Self-Sabotage Builder — Technical Report

Organization: NeuroGlitch · **© 2026 NeuroGlitch. All Rights Reserved.**

Document purpose: Single reference for engineering, production, and **Electron** desktop packaging. This describes how the game works today, how data flows, and how to wrap it without changing core gameplay or introducing lag.

Scope: Client-side game (`index.html` , `style.css` , `script.js` , `supabase-config.js` , `assets/`). Optional `server/` exists for legacy HTTP API; **the live game does not require it** for play, auth, or global leaderboard (Supabase is used from the browser).

---

1. Product summary

Self-Sabotage Builder is a single-player **canvas** game in two modes:

- **Build:** Place tiles on a large grid; validation enforces rules (e.g. one checkpoint, start/goal).
- **Play:** The same grid is played as a platformer. Each tile type has **sabotage** behavior (hidden until play, seeded per run, consistent within that run).

IP: Game title, mechanics, and branding are property of NeuroGlitch; founders and legal notice appear in `index.html` / `script.js` headers.

---

2. Architecture (web → Electron)

Layer	Implementation
UI	HTML modals, CSS themes, fixed-size **960×576** canvas
Game logic	One IIFE in `script.js` (~7k+ lines): input, physics, tiles, audio, save/load
Rendering	Canvas 2D, `requestAnimationFrame` single loop
Persistence	`localStorage` key `SSB_SAVE_V2` (players, levels, settings)
Online	Supabase Auth + Postgres (`profiles`, `leaderboard`, `user_completed_levels`) via `@supabase/supabase-js`

Electron mapping: The game is a **static site**. An Electron `BrowserWindow` loads `index.html` via `loadFile()` or `loadURL('file://...')` from the packaged `app.asar` or unpacked resources. **No change to game code is required** for basic packaging; the same bundle runs in Chromium.

---

3. Runtime stack

- **Canvas:** `#game` — `width="960" height="576"` in `index.html` .
- **World grid:** 64×36 tiles at 32 px/tile (`COLS` / `ROWS` / `TILE` in `script.js`); classic built-ins use a smaller legacy footprint placed in the world.
- **Main loop:** `requestAnimationFrame(frame)` ; `dt` clamped (e.g. 4–32 ms) for stable physics; **no** `setInterval` game tick.
- **Input:** `Input` class — `keydown` / `keyup` with normalized keys (WASD, arrows, space); `tick()` clears per-frame pressed/released after simulation.
- **Audio:** Web Audio API (procedural SFX, ducking, optional music/ambient per settings).

---

4. Game systems (concise)

4.1 Modes

- **Build:** Palette, budgets, undo/redo, copy/paste regions, draft autosave, validation HUD.
- **Play:** Player entity, tile runtime state, sabotage resolution, camera follow (smoothed), particles (capped pool), optional timer on preconfigured levels, checkpoint respawn.

4.2 Physics (representative)

Constants in `PHYS`: acceleration, max speed, friction, gravity, jump velocity, air control. Mud and powerups (speed boost, double jump, protection) modify multipliers. **Coyote time** and **jump buffer** improve input forgiveness. Horizontal movement includes air release friction tuning and ground turn assist.

4.3 Sabotage

Difficulty/sabotage intensity scales with validation difficulty and settings. Tile types (platform, spikes, jumppad, hex, lava, etc.) carry per-type sabotage payloads applied when entering play.

4.4 Progression

- Local **players** with stats, saved **levels**, featured samples, built-in catalog (tutorial / easy / medium / hard), random generator.
- Optional **challenges** (e.g. no double-jump, death limit) on start modal.

---

5. Data layer

5.1 Local (`localStorage`)

- Key: `SSB_SAVE_V2`.
- Contains players, active player id, settings (keybinds, theme, audio), and level data as designed by the game's save format.
- **Electron note:** `localStorage` is **per-origin**. For `file://`, storage can differ from `https://`; use a **stable custom protocol** or `loadFile` with a predictable origin (see §8) so saves and Supabase redirects behave consistently.

5.2 Supabase (optional)

- Config: `supabase-config.js` sets `window.SUPABASE_URL` and `window.SUPABASE_ANON_KEY` (**anon only**; never ship service role in the client).
- **Auth:** Email, magic link, password, anonymous — as enabled in Supabase dashboard.
- **Database:** SQL in `supabase/leaderboard_setup.sql` — `profiles`, `leaderboard` (score, level, XP, preconfigured completion counters), `user_completed_levels` (dedupe per built-in level), RLS policies.
- **Client:** Reads/writes from the renderer; leaderboard upserts occur **on wins**, not on idle page load (hydration reads only).

---

6. UI / UX highlights

- Device prompt (desktop vs mobile touch controls).
- Modals: start, levels, settings, leaderboard (global search + card layout, local list), auth/account.
- Themes and backgrounds; reduced-motion respect where styled.
- Touch: larger targets in mobile mode.

---

7. Electron integration (recommended practice)

7.1 Goals

- **Same game bundle** as the web build — no fork of `script.js` for "desktop only" unless you add desktop-specific features via **preload** IPC.
- **Stable performance:** default **hardware acceleration** on; avoid blocking the main process.

7.2 Minimal main process

- Create `BrowserWindow` with `webPreferences`:
  - `contextIsolation: true`
  - `nodeIntegration: false` in the **game** renderer (keep game logic browser-safe).
  - `preload` only if you need IPC (e.g. "Open user data folder", "Quit", updates).
- Load the game: `win.loadFile('index.html')` with files packaged under `resources/app/` or equivalent.

7.3 Security

- Do **not** enable `nodeIntegration` in the game window.
- Do **not** pass secrets via query strings; Supabase anon key is public by design but remains **anon** only.
- If using `webSecurity: false` for local file quirks, understand XSS risk; prefer `loadFile` + proper asset paths over disabling web security.

7.4 Storage & URLs

- Align **Supabase Auth** "Site URL" and redirect URLs with your Electron app's effective origin if you use OAuth/magic links.
- For consistent `localStorage`, avoid relying on changing `file://` paths; use a **registered protocol** (`app://`) or serve from `http://127.0.0.1` inside Electron if needed.

7.5 Updates & distribution

- Ship **asar** or unpacked static files; code-sign installers per your platform.
- Optional: `electron-updater` for auto-updates (outside this repo).

7.6 Lag avoidance

- Game already caps `dt` and uses a single rAF loop — **do not** add second animation loops in the renderer.
- Avoid synchronous Node APIs in the renderer; keep heavy work in main or worker threads if you extend the app.

---

8. Performance & stability (existing design)

- Single **update** → **render** pass per frame; input sampled once per frame after prior frame's `tick()`.
- Particle cap; debounced draft save; collision multi-pass to reduce tunneling.
- Tab **visibility** clears input state to avoid stuck keys after alt-tab (with window blur handling).

---

9. Deployment matrix

Target	Artifacts
Web static	`index.html`, `style.css`, `script.js`, `supabase-config.js`, `assets/`

itch.io / static host	Same; configure Supabase URLs
Electron	Same static bundle + Electron shell; sign & notarize per OS policy

---

10. Repository map (primary)

Path	Role
`index.html`	Shell, canvas, modals, script tags
`style.css`	Layout, themes, leaderboard UI
`script.js`	Entire game logic
`supabase-config.js`	Supabase URL + anon key
`supabase/leaderboard_setup.sql`	DB schema + RLS
`assets/`	Backdrop images, optional media
`server/`	Optional Express API (not required for core game + Supabase client)

---

11. Maintenance checklist

- After physics/UI tuning: regression test **built-in levels** (timers, tight jumps).
- After DB changes: run or diff `leaderboard_setup.sql` in Supabase SQL editor.
- For Electron: re-test **save persistence**, **auth redirect**, and **leaderboard** on a release build.

---

12. Glossary

Term	Meaning
Sabotage	Hidden per-tile behavior active only in play mode
Run	Single play session from spawn to win/lose
Coyote time	Grace period to jump after leaving ground
Jump buffer	Window where an early jump press still applies on landing

---

End of report. This document is intended to remain valid as a **future-facing** engineering reference; update version and date when the product or schema changes materially.

Generated for internal use — NeuroGlitch, 2026.

PDF output: `docs/Self-Sabotage-Builder-Technical-Report.pdf` — regenerate with: `python docs/generate_report_pdf.py` (requires `fpdf2`).